

A Formalization of the Mean-Field Derivation of the Vlasov Equation

Mathematician in the loop: AI-assisted Lean formalization as a strategy game

Joseph K. Miller*

July 13, 2026

Abstract

We formalize a research result in the Lean 4 proof assistant by having a mathematician direct an AI system, and frame the activity as a *formalization game*. The objective is to turn a $\text{\LaTeX}$ document into Lean. The game is *won* when the development compiles, contains no `sorry`, and a machine check shows the target theorems rest on Lean’s foundational axioms alone. Reuse is a second check, by a definition we introduce: whether the development yields a *self-contained layer* of general mathematics the wider library could absorb.

The case study is a complete, axiom-clean formalization of well-posedness for the nonlinear Vlasov equation via Dobrushin’s mean-field route — existence, uniqueness, the stability estimate and mean-field limit, and a short-window superposition principle (weak solutions are Lagrangian). The human’s role here was to direct, not to write proofs: to scope the definitions, steer the decompositions, and triage the library’s gaps; the AI agent executed. The formalization certifies the proof of each statement as written; whether the written statement is the intended theorem stays the mathematician’s judgment. The optimal-transport machinery that fell out of the build (in particular, properties of the Wasserstein-1 metric and the Kantorovich–Rubinstein duality theorem) separates into a self-contained layer (Definition 2.2) that compiles against Mathlib alone: about a sixth of the development (49 of 299 declarations), behind a 22-declaration interface with no reverse dependency. The headline theorems ran in about a week, the full development in about a month. We report the quantitative claims as observations of one game, not as general laws. The game’s rules name no particular system, so the methodological framing is meant to outlast the tools of any one run.

Contents

1	Introduction	2
1.1	Related work	3
1.2	Choosing the target	4
1.3	What we formalize	5
2	The formalization game	7
2.1	Objective, win condition, self-contained layer	7
2.2	The three phases	9
2.3	The strategy framework	11

*Stanford University, jkm314@stanford.edu; Massachusetts Institute of Technology, jkm314@mit.edu. The development is public. Source: https://github.com/Hydrodynamical/Vlasov_Meanfield_Formalization. Blueprint site and API documentation: https://hydrodynamical.github.io/Vlasov_Meanfield_Formalization/.

3	The mean-field derivation, formalized	13
3.1	From Newton to an exact weak solution	13
3.2	Wasserstein-1, in two faces	14
3.3	The dynamical core	15
3.4	The superposition principle, and why C^2	16
3.5	The reusability outcome	17
4	Assessment and outlook	18
4.1	The verification discipline	18
4.2	Timeframe and the division of labor	19
4.3	What generalizes	19
4.4	Limitations	20
4.5	Outlook	20
A	Formal statements	21
B	The standing instruction file: one lesson per series, verbatim	23

1 Introduction

A proof on paper is an argument that asks a reader to be convinced. A proof in a formal proof assistant is a different kind of object: one the computer checks. The definitions, the theorem, and the proof are written in a formal language, and the machine either accepts the proof or it does not. There is no room for “clearly,” and no referee to persuade. Lean is one such language. Over the past decade a community has assembled in it a large and growing library of mathematics, called Mathlib, on which new formal proofs can be built.

For most of its history, formalization has trailed far behind the mathematics it certifies. A result a mathematician reads in an afternoon can take a skilled formalizer weeks to put into Lean, and the obstacle is rarely the logic. It is the volume of small intermediate steps, the labor of matching an informal argument to the library’s conventions, and, in analysis especially, the parts of the library that do not yet exist. This is what has kept formal proof a specialist’s craft rather than an everyday tool.

Two things have recently changed: AI systems can now write Lean proofs, and they can write them quickly. This raises the possibility that formalization can keep pace with ordinary mathematical work. That speed sharpens two questions about what the machine produces: whether one can trust proofs one did not write, and whether anyone can use what is left behind. This paper turns both into machine checks. Trust is answered by a *win condition* — the development compiles, contains no `sorry`, and depends on Lean’s foundational axioms alone — which certifies a proof relative to the statement as written, leaving statement-faithfulness with the mathematician. Reuse is answered by a *self-contained layer* — a dependency-isolated body of general mathematics, behind a small interface, that builds against the ambient library alone — which certifies that what remains is absorbable rather than disposable. Both are stated precisely in Section 2 (Definitions 2.1 and 2.2).

We put both checks to the test by doing the work and reporting what it required. We formalized a genuine result from kinetic theory — Dobrushin’s derivation of the nonlinear Vlasov equation, a classical passage from the motion of many interacting particles to the equation governing their average — in Lean, over about a month. We did not write the Lean ourselves: a human mathematician directed an AI system, which wrote the proofs, while the mathematician decided what to

prove, how to cut each hard theorem into reachable pieces, and when to abandon a line that was not working.

The win condition was met here, and the development yields a self-contained layer. The first is discharged by the machine: `#print axioms` reports the target theorems depend on Lean’s foundations alone. What it cannot supply is the other half — that each statement as written is the theorem intended — nor the judgment that carried the proofs; the case study marks both, stage by stage. Getting the solution concept right (Section 3.1): the weak-solution test class had silently collapsed, leaving a key statement vacuously true behind a green build, caught by a human read. Building the distance (Section 3.2): the duality bridging its two faces was absent from the library, won by human-steered search and agent throughput. The stability estimate (Section 3.3): read in the dual face it appears to need an attainment theorem the library lacks; a human redirection, back to Do-brushin’s own coupling formulation, removed the need. The superposition principle (Section 3.4): one degree of regularity deliberately spent to make uniqueness go through — a human typeclass decision, the construction the agent’s. The layer is delivered in turn: the general mathematics the argument demanded separates cleanly from the problem-specific development (Section 3.5), fit for the ambient library. The human side of each unlock is not incidental — that judgment accumulates, in a standing guidance file built as play proceeds (Section 2.3).

1.1 Related work

This work borders three recent lines: the end-to-end formalization of known-hard results, the autonomous proof-search agents, and first-hand methodological accounts of AI-assisted mathematics.

The first is the now-established tradition of formalizing a substantial, known-hard mathematical result *end to end* in Lean: the Liquid Tensor Experiment [1], the polynomial Freiman–Ruzsa theorem [2], Carleson’s theorem on the convergence of Fourier series [3], and the sphere-eversion project [4] are the canonical examples. These share an infrastructure — Massot’s `leanblueprint` [5], a human-readable proof skeleton linked to the formal development, which the present project also uses — and a common mode: a human mathematician, or team, directs the formalization of a result whose mathematics is already understood. The present work is squarely in that lineage; what sets it apart is making the *strategy* of the human direction explicit — naming, phase by phase, what the mathematician is doing.

A parallel and recent line is *autonomous*. AlphaProof [6] couples reinforcement learning with Lean and reached silver-medal performance at the International Mathematical Olympiad; its agentic successor [7] and Harmonic’s Aristotle [8] drive evolutionary language-model/Lean loops and have autonomously resolved open Erdős problems [9]; LeanMarathon [10] is a multi-agent, blueprint-centric harness that autoformalizes the theorems of recent Erdős-problem papers; and Alexeev, Lichtman, Tao, and coauthors [11] resolved open Erdős conjectures on primitive sets with a key method suggested by output of GPT-5.4 Pro and the results formalized in Lean, in part by Math Inc.’s autonomous Gauss. The present work runs at the other end of the autonomy axis: the mathematician directs throughout, and *documenting* that direction — where to cut a hard theorem, which routes the ambient library can support, when to abandon one — is itself a facet of the paper. The autonomous systems do exhibit exactly the failure mode the win condition (Definition 2.1) is built to catch — a renamed `sorry`, a hallucinated lemma cited as settled — documented by the systems’ own authors [7].

Closest to this paper are *first-hand* methodological accounts of AI-assisted research mathematics. Ilin [12] semi-autonomously formalized a Vlasov–Maxwell–Landau *equilibrium* in Lean 4 under a single supervising mathematician (using, among other tools, Claude Code and Aristotle) — the same domain and broadly the same mode as here, but a static characterization rather

than the dynamic well-posedness, stability, the limit, and superposition principle [21] we formalize, and without the methodological apparatus we foreground. In mathematical physics more broadly, Douglas, Hoback, Mei, and Nissim [13] formalized the free bosonic quantum field theory in four dimensions against the Glimm–Jaffe axioms in Lean 4, as a proof of concept for AI-assisted formalization of extended mathematical-physics arguments; their original release assumed three classical results (Minlos’ theorem among them) as disclosed placeholders — the deferred gap of Section 2.2, since discharged. Contemporaneously, Armstrong and Kempe [14] formalized the core interior De Giorgi–Nash–Moser regularity theory for uniformly elliptic divergence-form equations in Lean 4 — to their knowledge the first machine-checked formalization of a major theorem in modern PDE regularity — likewise carried out with extensive use of large language models under close human supervision. Their account, like ours, locates the decisive human contributions in detailed proof blueprints, careful interface design, and validation against the checked artifact. Zimmer et al. [15] encode agentic-research norms as prompt “commandments,” a close parallel to our standing-instruction file; and Avigad [16] argues that mathematicians should actively deploy and shape these tools rather than merely react to them — a call to which this paper is a concrete response. What distinguishes the present work against this backdrop is the target it runs on: a *dynamical* result for a nonlinear PDE — well-posedness, stability, the mean-field limit, superposition — carried end to end as one development.

1.2 Choosing the target

Derivations of this kind come in a range of difficulty, set by how rough the force between particles is. Where the force is smooth, the passage to the limiting equation is classical, going back to Dobrushin [18] and the Braun–Hepp–Neunzert line [19, 24] (with the later surveys [22, 23]). Where the force is singular, as in gravitation or electrostatics, the same passage is a celebrated open problem, settled so far only in softened cases [25, 26, 27]. We formalize the smooth end, and chose it deliberately: it is where a reusable library can be built. Closing the singular frontier is a problem of mathematics, not of proof engineering — the missing ingredient is new analysis, and the most a mature library could offer there is a place to check such arguments as they are found. The library amplifies the mathematician; it does not stand in for the mathematics.

We took it less for any one theorem than as a faithful representative of its kind: its stages — the empirical distribution of the particles, a notion of distance between such distributions, the flow that moves them, and the equation they satisfy in the limit — recur across the subject. Writing it in Lean meant first building general mathematics the library does not yet contain, about distances between probability distributions and the flows that move them. To our knowledge no prior formalization had assembled it; this is the self-contained layer of Definition 2.2, and the lasting artifact of the exercise.

We describe the activity as a *formalization game* because the framing is precise enough to be load-bearing. There is a definite objective (turn the $\text{\LaTeX}$ into Lean), a machine-checkable win condition (no sorry, a clean axiom footprint), and a payoff beyond winning — whether the won development yields a *self-contained layer*. And there are three phases, each rewarding a different strategy: a player who opens well can still lose the endgame. The framing is also built to outlast its tools: the rules name no particular system and no division of labor, so what was attempted and what was won stays meaningful as the machines underneath turn over (Section 4).

An alternative notion of solution of (2) is a *Lagrangian* solution: a weak solution transported by its own characteristic flow.

Definition 1.2 (`IsLagrangianVlasovSolution`). A *Lagrangian* solution is a weak solution $f : [0, T] \rightarrow \mathcal{P}_1(\mathbb{R}^d \times \mathbb{R}^d)$ for which there exists a flow $(X, V) : [0, T] \times (\mathbb{R}^d \times \mathbb{R}^d) \rightarrow \mathbb{R}^d \times \mathbb{R}^d$ solving the characteristic system

$$\dot{X}(t, z) = V(t, z), \quad \dot{V}(t, z) = -(\nabla W * \rho_t)(X(t, z)), \quad (X, V)(0, z) = z,$$

such that $f_t = (X(t, \cdot), V(t, \cdot))_{\#} f(0)$ for every $t \in [0, T]$. Here the pushforward $\Phi_{\#}\mu$ of a measure μ by a Borel map Φ is $(\Phi_{\#}\mu)(A) = \mu(\Phi^{-1}(A))$, equivalently $\int \varphi d(\Phi_{\#}\mu) = \int \varphi \circ \Phi d\mu$ for every bounded continuous φ .

The Lean predicates are leaner than the prose: `IsVlasovSolution` states exactly the differentiability and the derivative identity, for a curve of measures over all of $\mathbb{R}$, and `IsLagrangianVlasovSolution` adds the flow witness (Appendix A); membership in $\mathcal{P}_1$, the window, and continuity in time are imposed where they are consumed, as explicit hypotheses of the theorems. Theorem 1.7 quantifies over the windowed variant `IsVlasovSolutionOn`, with continuity in time tested against compactly supported continuous functions — equivalent to narrow continuity for probability-valued curves.

Every Lagrangian solution is weak, by definition. The converse — that every weak solution is Lagrangian — is the substantive direction: the classical superposition principle [21]. We prove a short-window version under the strengthened assumption `AssW2` introduced below (Theorem 1.7; Section 3.4).

Theorem 1.3 (Global well-posedness, `vlasovWellPosedness`). *Let W satisfy Assumption 1 and $f_0 \in \mathcal{P}_1(\mathbb{R}^d \times \mathbb{R}^d)$. The forward Cauchy problem is well-posed: there exists a single curve $f : [0, \infty) \rightarrow \mathcal{P}_1(\mathbb{R}^d \times \mathbb{R}^d)$ with datum $f(0) = f_0$ and finite first moment at every $t \geq 0$, which is a Lagrangian solution of (2) on every window $[0, T]$, in the sense of Definitions 1.1 and 1.2; and on each window it is the unique such Lagrangian solution.*

The single global curve is `vlasovWellPosedness`; per-window uniqueness in the class of Definition 1.2 is `vlasovWellPosedness_uniqueness`, a separate statement. It follows for $L > 0$ from the stability estimate below; the degenerate $L = 0$ case (constant force) is explicit, so uniqueness holds at every L .

Solutions of (2) depend stably on their data; the relevant distance is the Wasserstein-1 (Kantorovich–Rubinstein) metric on $\mathcal{P}_1$.

Definition 1.4 (`wasserstein1`). For $\mu, \nu \in \mathcal{P}_1(\mathbb{R}^d \times \mathbb{R}^d)$,

$$W_1(\mu, \nu) := \sup_{\text{Lip}(f) \leq 1} \left(\int f d\mu - \int f d\nu \right) = \inf_{\pi \in \Pi(\mu, \nu)} \int \text{dist}(z_1, z_2) d\pi,$$

the supremum over 1-Lipschitz $f : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ and the infimum over the set $\Pi(\mu, \nu)$ of *couplings* — probability measures on the product with marginals μ and ν . The supremum is the *dual* face, the infimum the *primal*; they agree by Kantorovich–Rubinstein duality [20, 21].

Both faces and the duality between them are proved in the development: the easy direction (dual $\leq$ primal) is `wasserstein1_le_wasserstein1_coupling`; the equality, for probability measures with finite first moment on a Polish space, is `wasserstein1_eq_coupling`, whose hard direction is built in Section 3.2. Neither optimum is attained anywhere — every bound is ε -optimal.

Theorem 1.5 (Dobrushin stability [18], [dobrushin](#)). *Let W satisfy Assumption 1. Any two Lagrangian solutions f, g with finite first moment at every time are stable in W_1 at an exponential rate,*

$$W_1(f_t, g_t) \leq e^{Ct} W_1(f_0, g_0), \quad t \geq 0, \quad C = C(L).$$

Applied with g the empirical measure of the Newton dynamics (1), the estimate gives the mean-field limit as a corollary.

Corollary 1.6 (Mean-field limit, [meanFieldLimit](#)). *Let f be the Vlasov solution with datum f_0 , let μ_t^N be the empirical measure of N particles evolving by (1), and assume the estimate of Theorem 1.5 holds for every pair (μ^N, f) with one constant C . If the initial empirical measures converge, $W_1(\mu_0^N, f_0) \rightarrow 0$ as $N \rightarrow \infty$, then for every $T > 0$*

$$\sup_{t \in [0, T]} W_1(\mu_t^N, f_t) \xrightarrow[N \rightarrow \infty]{} 0.$$

Theorem 1.5 supplies the assumed estimate — each empirical curve is a Lagrangian solution with finite moments, and the rate $C = 2 \max(1, L)$ of Section 3.3 is uniform in N — but the Lean takes it as an explicit hypothesis rather than re-deriving it (Appendix A).

Extending stability and uniqueness from Lagrangian solutions to *all* weak solutions is the superposition principle; the version we prove costs one degree of regularity beyond Assumption 1 and runs on a short window.

Assumption 2 ([Assw2](#)). In addition to Assumption 1, let $\nabla W \in C^1$ (equivalently $W \in C^2$).

Theorem 1.7 (Superposition principle, [weak_isLagrangianVlasovSolutionOn](#)). *Let W satisfy Assumption 2 and let $T > 0$ satisfy $LT^2 < 1$. Every weak solution $f : [0, T] \rightarrow \mathcal{P}_1(\mathbb{R}^d \times \mathbb{R}^d)$ whose first moments are uniformly bounded on $[0, T]$ and whose force field is regular — $(\nabla W * \rho_t)(x)$ continuous in t for each x , its spatial derivative jointly continuous on $[0, T] \times \mathbb{R}^d$ — is Lagrangian on $[0, T]$.*

The hypotheses, reproduced in full in Appendix A, are not cosmetic: the window is short, and the continuity hypotheses concern the solution’s *own* force field. Chaining short windows to arbitrary T , as the well-posedness construction does for its flow, is not formalized for this bridge; it is future work.

This introduction has stated the two checks — the win condition and the self-contained layer — set down in mathematics exactly what we formalized, and placed the work among its neighbors; the rest of the paper is the method and its evidence. Section 2 sets out the game — its win condition, its payoff (the self-contained layer), the three phases, and the L/P/M/B framework of standing guidance the human accumulates. Section 3 is the heart of the paper: Dobrushin’s derivation, run as a formalization in its mathematical order — from the particle system to the limiting equation, through the core estimates, to the gaps where the library ran out — marking at each stage where the mathematics was hard and who unlocked it. Section 4 assesses the method: the discipline that kept it sound at speed, the timeframe and division of labor, what generalizes, and the limits of one run, closing with the next target and with what should outlast the tools.

2 The formalization game

2.1 Objective, win condition, self-contained layer

The substance of the *formalization game* is its win condition and the self-contained layer a win can export. It is played on three objects: a source mathematics document (a $\text{\LaTeX}$ paper), a

Lean 4 development $\mathcal{L}$ under construction, and a standing instruction file $\mathcal{G}$ governing the agent’s behavior. A distinguished set of *target theorems* $\Theta \subseteq \mathcal{L}$ encodes the main results of the source, and the human and the agent alternate moves that edit $\mathcal{L}$ and $\mathcal{G}$.

Definition 2.1 (Win condition). The game is *won* when:

- (W1) $\mathcal{L}$ compiles, with no occurrence of `sorry` in any declaration reachable from Θ ; and
- (W2) for every $\theta \in \Theta$, the command `#print axioms  $\theta$` returns exactly `[propext, Classical.choice, Quot.sound]`.

The second clause is the substantive one. A development can be free of `sorry` and still lean on a bespoke `axiom` — a `sorry` renamed to look principled. `#print axioms` traces the *complete* axiom set a theorem depends on; a footprint of exactly `propext`, `Classical.choice`, and `Quot.sound` — Lean’s own foundations — means proved modulo those alone, and any fourth name is a loss a build-success check would miss. The win condition is “compiles *and* the foundations are clean,” not “compiles.”

This certifies a theorem soundly *relative to its statement as written*; it does not certify that the written statement is the theorem intended. That second responsibility, statement-faithfulness, is the human’s, discharged in the early game through definitional scoping and atom-level signature reading.

We define reusability as the existence of a *self-contained layer*: a subset of the development the ambient library could absorb with no trace of the problem it came from, checked pass/fail by criteria the library defines, not us.

Definition 2.2 (Self-contained layer). A won game exports one or more *self-contained layers*. A subset $\mathcal{L}_{\text{gen}} \subseteq \mathcal{L}$ is *self-contained* when it passes four checks, each re-runnable by a referee and each defined by the ambient library rather than by us:

- (Q1) **Isolation.** In the elaboration environment’s declaration-level dependency graph, no edge runs from $\mathcal{L}_{\text{gen}}$ into $\mathcal{L} \setminus \mathcal{L}_{\text{gen}}$: every cross-edge points inward, through an interface of width w (the number of $\mathcal{L}_{\text{gen}}$ declarations referenced from outside). The reverse-edge count is 0, read directly off the graph.
- (Q2) **Standalone compilation.** $\mathcal{L}_{\text{gen}}$ builds as a package depending only on the Mathlib library (v4.29.1), with zero problem-specific files in its import graph. This is the compiled certificate of (Q1): the build cannot succeed if a hidden reverse edge exists.
- (Q3) **Linter-clean.** $\mathcal{L}_{\text{gen}}$ passes the ambient library’s own linter suite with zero warnings — the automated gate its maintainers apply before any merge (unused hypotheses, simp-normal-form, naming, docstring coverage).
- (Q4) **Non-redundant.** No declaration in $\mathcal{L}_{\text{gen}}$ restates a result already present in the ambient library, and every declaration is reachable from a target theorem — no duplication of existing mathematics, no dead code.

A layer passing (Q1)–(Q4) is *self-contained*, separated from the problem-specific development through an interface of width w . The *size* of $\mathcal{L}_{\text{gen}}$, as a fraction of $\mathcal{L}$, is reported descriptively, not as a criterion. We claim reusability in this precise, certified sense, and distinguish it from demonstrated *reuse*, which we do not assert.

General object (mathematics)	Lean declaration
Wasserstein-1, dual (Kantorovich–Rubinstein) form	<code>wassersteinCost, wasserstein1</code>
Wasserstein-1, primal (coupling) form	<code>wasserstein1_coupling</code>
Truncated variant $\bar{W}$ (bounded ground cost)	<code>wassersteinBar</code>
Dual = primal (Kantorovich–Rubinstein bridge)	<code>wasserstein1_eq_coupling</code>
Finite Kantorovich duality (from Farkas)	<code>finiteRange_transportation_dual</code>
Coupling-cost triangle inequality (gluing)	<code>wassersteinCost_coupling_triangle</code>
Non-expansion under Lipschitz pushforward	<code>wasserstein1_pushforward_le_iInf</code>

Table 1: Representative contents of the self-contained layer $\mathcal{L}_{\text{gen}}$ — general optimal-transport mathematics that fell out of the build, stated once and reused. The full layer is 49 declarations behind a 22-declaration interface, compiling against Mathlib alone.

In the case study, $\mathcal{L}_{\text{gen}}$ is the optimal-transport machinery built for the derivation — the Wasserstein-1 metric in both faces, the Kantorovich–Rubinstein bridge between them, a finite Kantorovich duality, and the coupling-gluing triangle inequality (Table 1). It passes all four checks; the numbers — interface width, layer size, and the reverse-edge count — are reported with the outcome in Section 3.5.

The three phases reward *different* strategies: a player who opens well can still lose the endgame. The phase structure, not the win condition alone, is the substance of the game.

2.2 The three phases

2.2.1 Early game: definitional scoping

In the early game the player translates the source’s target theorems into Lean *statements*, proofs left as `sorry`. The difficulty is not the statements but the *definitions* they quantify over: every later move builds on them, so a definition that bakes in the wrong hypothesis, metric, or regularity class propagates its error through the whole development and is most expensive to find late.

The early-game risk is definitional. The dominant strategy is to lock the object scaffolding — the definitions and the target signatures — before proving anything, and to read the source at the level of atoms (which exact hypothesis, which exact moment, which exact norm), not of prose.

The player commits the statements and their definitions, compiles with every proof a `sorry`, and only then proves. This “API-lock” separates committing to an interface from discharging it — and the interface is what every later phase is written against, so errors caught here are cheap and the same errors caught in the endgame are not.

2.2.2 Mid game: steering the decomposition

In the mid game the target’s sorried statement is repeatedly *decomposed* into smaller sorried sub-statements, each heuristically easier, until the leaves are within the agent’s reach. The splitting itself is delegable — a *sorry-decomposer* sub-agent does exactly that (Section 3) — which is what isolates the mid game’s real demand: **steering was the part of the loop that most demanded human judgment**. The agent could prove a well-specified sub-lemma and split an oversized one; what it did not supply was the judgment of *where* to cut, which of several equivalent routes the library could support, and when to abandon a route — acts of mathematical taste. The observation

is time-stamped (this system, this moment; autonomous steering is exactly the frontier systems improve at fastest), not a permanent division of labor. In the case study the decisive steering was of this kind — which face of the distance to argue the stability estimate in, a call that meant returning to the source’s own formulation, and which form of its underlying inequality the encoding could support (Section 3.3).

This is also where the loop *learns*: a recurring pattern — more often a recurring *failure* — is distilled into a strategy and written into $\mathcal{G}$, which is what makes the standing instruction file a live part of the game rather than a fixed prompt.

The mid-game risk is choosing a decomposition the library cannot support, or one that multiplies rather than reduces difficulty. The dominant strategy is human-steered decomposition — cut where the mathematician sees tractability — combined with encoding each hard-won heuristic into $\mathcal{G}$ so it is reused, not re-derived.

2.2.3 End game: triaging the library gaps

Eventually the surviving sorries are no longer decomposable into project-specific pieces: they are requests for *general* mathematics the ambient library may or may not contain, and the endgame is the triage. Each falls into one of three kinds. A *phantom* looks like a gap but dissolves under a sharper reading of what the consumer needs — the case study had one, a conjectured metrization the development never used (Section 3.3). A *genuine gap you build* is standard mathematics the library lacks that the player formalizes from the primitives present — here, a finite-dimensional duality theorem. A *genuine gap you defer* is marked with an explicit, cited placeholder when the cost of building outweighs the value and the dependency is a recognized, true theorem — here, a Polish-space completeness fact for the Wasserstein metric, deferred off every certified path.

The end-game risk is misclassifying: building infrastructure for a phantom, or silently assuming a gap that should be marked. The dominant strategy is triage against the library’s actual coverage — which requires knowing the library at the functional-analysis level — and verifying each candidate gap at the level of what the consumer truly needs before committing to build it.

Progress in formalizing research analysis is rate-limited by the ambient library’s infrastructure; the endgame is the work of locating, and either filling or honestly marking, where it runs out.

The case study of Section 3 left a complete, machine-readable record: every move is a version-control commit, so the game’s state — its `sorry` count, its standing instruction file $\mathcal{G}$, its file structure — can be reconstructed at any point of its 362-commit history.¹ Three conventions fix what the figures count: “live `sorry`” counts the token after stripping Lean line- and block-comments (the word appears constantly in prose and docstrings, so an unstripped count is meaningless; the count reaching 0 at the final commit, where the build is independently known clean, validates the strip); the production library is the Lean package source, excluding the throwaway development scratch file; and the commit classification of Section 4.2 is a heuristic over commit-message intent.

¹Reproducible scripts and the raw data are in the development’s `formalize/retrospective` directory.

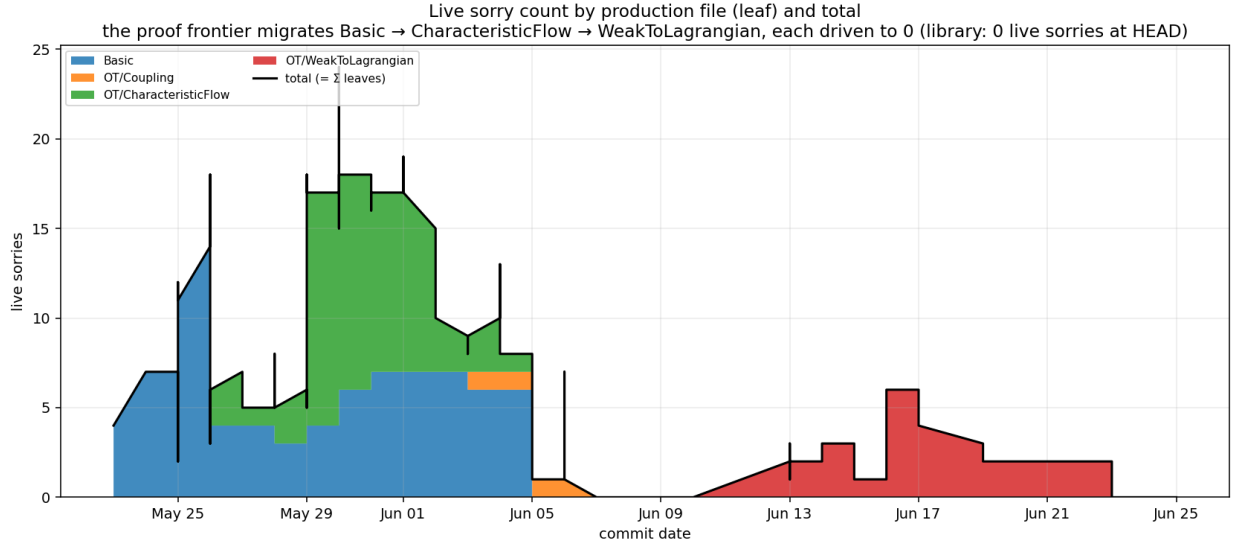


Figure 2: From the case study of Section 3 (362 commits): live `sorry` count per production file, stacked so the leaves sum to the total (black line). The open-goal front is concentrated in one active leaf at a time and migrates across the development; the total peaks at 24 and returns to 0. The two humps are two games — the well-posedness/stability marquee and the superposition-principle follow-on — separated by the zero-`sorry` reusability extraction.

Three series, read off that record, turn the claims of this section and the next into data. The first is the `sorry` front: Figure 2 plots the live count per source file, and the three-phase rhythm is visible in it directly. Open goals stay concentrated in a single *active leaf* — one production file at a time carries the bulk of the `sorries`, and the front migrates from the basic objects, through the characteristic-flow well-posedness core, to the weak-to-Lagrangian bridge, each interface locked and its goals driven down before the front advances: the early-game API-lock discipline made visible. The total never runs away, because the game is a sequence of bounded burndowns, not one accumulating debt; and the flat stretch between the figure’s two games, at zero live `sorry`, is the reusability extraction of Section 3.5, banking the layer without reopening a goal.

2.3 The strategy framework

The standing instruction file accumulated its lessons in four series, which operate at genuinely different levels with different triggers for “this rule applies.”

L-series — Lean / agent-tool lessons.

Idioms, elaborator quirks, parser quirks, and the engineering of agent sub-task specifications. Consulted when writing or debugging an individual Lean proof.

P-series — Process discipline.

When to ask, commit, delegate, pivot, and how to verify. Consulted at the per-move discipline level.

M-series — Mathematical structure.

Which mathematical structures to work in, and what shape a proof should take. Consulted when designing a proof.

B-series — Bridging architecture.

When to build new infrastructure versus patch an existing seam. Consulted when frictions accumulate across attempted bridges.

One representative lesson from each series — earned, and reproduced verbatim from $\mathcal{G}$ — appears in Appendix B.

The lessons were not designed in advance but *earned* — each keyed to a specific failure that occurred during the game and was then encoded into $\mathcal{G}$ to prevent its recurrence. The standing instruction file records the mathematician’s accumulated strategic experience in a form the agent can consult, amortizing the human’s judgment across the thousands of moves the agent makes.

The second series shows this accumulation as data: Figure 3 plots the count of named strategies in $\mathcal{G}$ across its 36 revisions, split by series. Two landmarks are legible: the four-series taxonomy crystallizes in a single revision (before it, an undifferentiated list of tool lessons; after, all four series present), and the process series jumps by four rules at once, in the aftermath of exactly the cascade of frictions the three phases above describe. The closing growth is the tool-lesson series alone: through the superposition follow-on it is Lean idioms that accumulate, not new strategy, so the file tracks the novelty of the tactical terrain crossed rather than the size of the proof obligation.

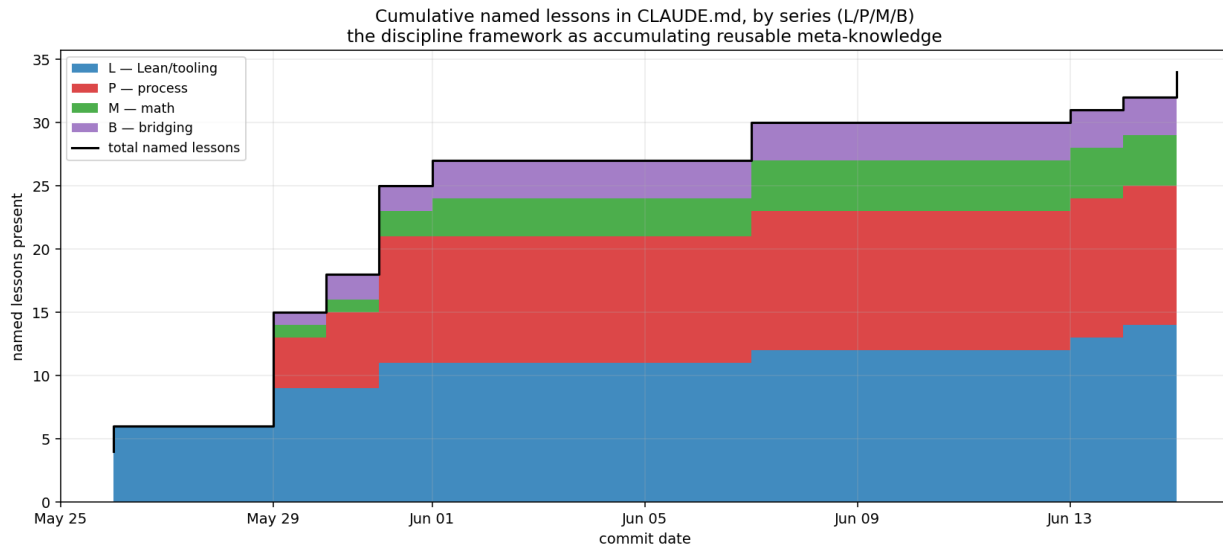


Figure 3: From the case study of Section 3: named strategies present in the standing instruction file $\mathcal{G}$ across its 36 revisions, by series (L/P/M/B). The staircase shape — flats punctuated by jumps at specific failures — is the signature of strategies earned rather than designed. The total grows 4 $\rightarrow$ 34; the four-series taxonomy crystallizes in one revision, and the process (P) series jumps by four in the aftermath of the cascade frictions of Section 2.2.

3 The mean-field derivation, formalized

We played the game on Dobrushin’s derivation [18] of the Vlasov equation from N -particle Hamiltonian dynamics; for the broader mean-field theory see [22, 23, 24]. This section replays the derivation in its mathematical order — get the objects right, build the distance, run the dynamical core, reconcile the two solution notions, extract the layer; one subsection each — and at each stage marks the two things the finished proof hides: where the mathematics was genuinely hard, and who unlocked it. The answer differs by stage — a human read, human-steered agent throughput, a human redirection, both at once — and that spread, visible in the derivation rather than asserted about it, is the division of labor. The coupling argument of Section 3.3 is shown in Dobrushin’s terms, at paragraph length — it is where the optimal-transport machinery is spent. The endgame’s library-building — the general mathematics the ambient library lacked — is marked where it arises rather than gathered into a section of its own.

The agent system was Claude Code, Anthropic’s agentic coding environment, driving a frontier Claude model — Opus 4.7, then 4.8, as the build spanned the upgrade. The human and the model alternated moves in an interactive session, and mechanical, well-scoped sub-tasks were delegated to specialized sub-agents — a *sorry-prover* (close a single `sorry` by search against the local Mathlib), a *sorry-decomposer* (split one oversized `sorry` into named helper lemmas), and library-scout and verifier agents — coordinated by a small driver script; these delegated sub-agents ran on `claude-sonnet-4-6` (recoverable from the project’s agent logs). The standing instruction file $\mathcal{G}$ of Section 2.3 is the project’s in-repository `CLAUDE.md`. The development is a public git repository, from which the exact model versions, the full standing instructions, and the per-session agent logs are recoverable.

3.1 From Newton to an exact weak solution

Hard: getting the solution concept right. Unlocked by: a human read of the test class.

The derivation opens with N identical particles in $\mathbb{R}^d$ under the mean-field scaling, governed by the Newton equations

$$\dot{x}_i = v_i, \quad \dot{v}_i = -\frac{1}{N} \sum_{j \neq i} \nabla W(x_i - x_j), \quad i = 1, \dots, N,$$

where the pair potential $W : \mathbb{R}^d \rightarrow \mathbb{R}$ is the section’s standing assumption (the type class `AssW`): continuously differentiable, even, with globally Lipschitz gradient of constant $L := \text{Lip}(\nabla W)$. The $1/N$ coupling holds the kinetic and interaction energies at the same order as $N \rightarrow \infty$, and the bridge to the kinetic picture is the *empirical measure* $\mu^N := \frac{1}{N} \sum_i \delta_{(x_i, v_i)}$, a probability measure on the phase space $\mathbb{R}^d \times \mathbb{R}^d$ — in Lean, `empiricalMeasure` on `PhaseSpace d`.

The mathematics of the opening is standard bookkeeping — and, fittingly, among the first sorries the agent closed. Differentiating $\langle \mu_t^N, \varphi \rangle$ along the Newton flow and *adding and subtracting the diagonal* $i = j$ produces the convolution against the spatial marginal plus a diagonal remainder, which the evenness of W (part of `AssW`) kills: $\nabla W(0) = 0$, so the empirical measure solves the weak equation *exactly*, not merely to $O(1/N)$. Even here the win condition leaves a mark — a remainder may not be discarded silently, so the statement (`weakEvolutionEmpiricalMeasure`) returns it explicitly, its value and bound as conjuncts, and evenness collapses it in the corollary `empiricalMeasureSolvesVlasov`.

What was not routine is the definition the theorem quantifies over. The weak-solution predicate ranges over the C^∞ compactly supported test functions:

```
def IsVlasovSolution (gradW : PhysSpace d → PhysSpace d)
  (f : ℝ → Measure (PhaseSpace d)) : Prop :=
  ∀ φ : PhaseSpace d → ℝ, ContDiff ℝ (τ : ℕ) φ → HasCompactSupport φ →
  ∀ gradXφ gradVφ : PhaseSpace d → PhysSpace d,
    (∀ z, gradXφ z = gradient (fun x => φ (x, z.2)) z.1) →
    (∀ z, gradVφ z = gradient (fun v => φ (z.1, v)) z.2) →
    WeakEvolutionEq gradW f φ gradXφ gradVφ (fun _ => 0)
```

Its first version was *definitionally wrong*, in a way that collapsed every statement quantifying over it to a triviality. The error was ours, and it is the early game’s own point: getting a definition right at the start is hard, and the wrong version typechecks. In the Mathlib we build against, the top exponent in `ContDiff ℝ τ` means *real-analytic*, not C^∞ (the meaning of τ had changed earlier in the library’s history; the stale idiom still compiles); a nonzero real-analytic function cannot have compact support, so the test class had quietly collapsed to $\{0\}$ and the weak-solution hypothesis was *vacuously* true. The build stayed green throughout: the axiom footprint certifies a proof relative to the statement as written, and cannot see that the statement had become empty. A thirty-second `#check` caught the collapse, and the fix (`ContDiff ℝ (τ : ℕ)`, the C^∞ element written above) restored the class at twelve sites — the clearest instance in the development of the boundary Definition 2.1 draws.

3.2 Wasserstein-1, in two faces

Hard: the bridge between the two faces. Unlocked by: human-steered search, agent throughput.

The distance that organizes everything to follow is the Wasserstein-1 metric W_1 (Definition 1.4). The development carries it in two faces. The *dual* face is the definition: a supremum over Lipschitz test functions, stated once over a general ground cost c and specialized to $c = \text{dist}$:

```
def wassersteinCost (c : α → α → ℝ) (μ ν : Measure α) : ENNReal := -- DUAL
  ⌊ (f : α → ℝ) ( _ : ∀ x y, |f x - f y| ≤ c x y),
    ENNReal.ofReal (∫ x, f x ∂μ - ∫ x, f x ∂ν)
def wasserstein1 (μ ν : Measure α) : ENNReal :=
  wassersteinCost (fun x y => dist x y) μ ν
```

The *primal* face is an infimum over couplings of (μ, ν) (Definition 1.4), and Kantorovich–Rubinstein duality equates the two faces at $c = \text{dist}$:

```
def wasserstein1_coupling (μ ν : Measure α) : ENNReal := -- PRIMAL
  ⌊ (π : Measure (α × α)) ( _ : IsCoupling π μ ν), ∫ z, edist z.1 z.2 ∂π
theorem wasserstein1_eq_coupling ... : -- KR duality
  wasserstein1 μ ν = wasserstein1_coupling μ ν
```

The two faces are easy to write down; the *bridge* between them had to be built, and it is the reusable layer’s most substantial construction. Its two pieces: the coupling cost’s triangle inequality (`wassersteinCost_coupling_triangle`), which disintegrates a coupling of (ρ, ν) over ρ and glues — the load-bearing, error-prone part being not the cost bound but that the glued measure has the *correct marginals*, exactly where a “near-citation” from the library hides a gap; and the hard direction

of the duality on finitely-supported measures (`wassersteinCost_coupling_le_dual_of_finiteRange`), which the Mathlib version we build against (v4.29.1) does not carry. The duality entered the development as a marked, `sorry`’d foundation and had to leave as the endgame’s built gap (Section 2.2): the win condition certifies nothing while the mark stands. The search for its proof was the mathematician’s to steer. The natural Hilbert-space form of the separation was assigned first, ran into systemic friction with the inner-product instances on the Euclidean type, and was called off; the appealing Birkhoff-vertex shortcut was scouted and called off (the library’s Birkhoff theorem is uniform-marginal; the transportation polytope here is not); the third assignment survived — the geometric separation on the plain product $\iota \rightarrow \mathbb{R}$, assembling Mathlib’s Farkas lemma (conic separation) [17] with two compactness facts proved by hand (primal attainment on the transport polytope; closedness of the image cone, the hypothesis Farkas consumes) and a c -transform folding the dual pair into a single admissible potential — an *inequality*, primal $\leq$ dual, with no attained optimum anywhere.

Unlike the stages that turned on a single decision, this bridge was won by a directed search plus throughput: the mathematician fixed the target, assigned each avenue, and called each abandonment; the agent did the sustained proof-work — some twenty-five commits assembling Farkas, the compactness facts, and the c -transform into the duality `wasserstein1_eq_coupling`, the single optimal-transport line `dobrushin` consumes. It is the clearest case in the development of the machine carrying a hard, unglamorous build to the end.

3.3 The dynamical core

Hard: the stability estimate. Unlocked by: a human redirection — back to Dobrushin’s own coupling formulation.

The limit equation is the nonlinear Vlasov equation, solved by the characteristic flow $\dot{X} = V$, $\dot{V} = -(\nabla W * \rho_t)(X)$ transporting the datum f_0 , with ρ_t the self-consistent spatial marginal — the nonlinearity: the force is generated by the very solution it transports. Its two marquee facts are forward well-posedness at arbitrary Lipschitz constant L (Theorem 1.3), a Banach fixed point in W_1 , and Dobrushin’s estimate (Theorem 1.5), whose proof drives the mean-field limit. Both are won (Definition 2.1).

A statement-level decision shaped the well-posedness theorem before any proof. Dobrushin’s is a *forward* Cauchy problem, and the marquee is posed that way — existence on $[0, \infty)$ — rather than as an $\exists!$ over all of $\mathbb{R}$. The two-sided claim was too strong: it forced backward-time machinery the evolution does not use. Uniqueness survives exactly where the mathematics puts it — per window, in the Lagrangian class (`vlasovWellPosedness_uniqueness`, Theorem 1.3) — so nothing is lost by proving forward. Matching the statement to what the argument supports is the early game’s other discipline, dual to definitional scoping.

Theorem 1.3 holds at *arbitrary* L , and reaching that was a matter of the *proof*, not the statement. Dobrushin’s global-in-time argument, tiled into short windows with a fixed unit force-window, forces L small ($L < 1$) — an artifact of the tiling, not of the mathematics, since Dobrushin’s theorem holds for every L . Tracing the dependency to its single structural origin (the window’s unit width, not any analytic mechanism — the artifact-versus-genuine distinction of Appendix B) gave the fix: *exact* tiling, windows of width exactly T/N so the chain lands on the target, killing the reach-slack a naive offset-drop would leave. The removal was certified by instantiating the theorem at $L = 2$, a formerly forbidden value, and confirming it typechecks with no smallness hypothesis.

The coupling argument. The decisive move is a redirection. Read in the dual W_1 , the estimate appears to need an *optimal* coupling of the data, an attainment theorem the library does not have. The mathematician’s call was to return to the source: Dobrushin states the estimate in the coupling

cost from the start — his metric *is* an infimum over couplings [18] — and there it needs only that the infimum lie below *any* particular coupling’s cost, no attainment. The route is his; the judgment was the pairing of library gap with source formulation.

Fix *any* coupling π of the data, optimal or not. Each solution is transported by its own characteristic flow, so pushing π forward by the pair of flows couples (f_t, g_t) , and `wasserstein1_pushforward_le_iInf` bounds

$$W_1(f_t, g_t) \leq \int \text{dist}(\Phi_f^t z_1, \Phi_g^t z_2) d\pi(z_1, z_2) =: Q_t,$$

with Q_0 the cost of π itself. Dobrushin’s two estimates close the bound [18], transplanted from his truncated metric to the unbounded W_1 with moment control: the force mismatch is controlled by the very quantity under estimate (`convolveFunctionMeasure_lipschitz_in_x`), the trajectory gap obeys the integral-form bound (`flow_difference_mild_bound`), and Grönwall (`gronwall_mild_le`) closes them to $Q_t \leq Q_0 e^{2Mt}$ with $M = \max(1, L)$ — the rate $C = 2 \max(1, L)$ of Theorem 1.5, the factor 2 not cosmetic but drift and forcing each contributing M . The mild form is Dobrushin’s own, and it is the form the encoding can support: Grönwall on $t \mapsto W_1(f_t, g_t)$ directly — the development’s first route — demands a time-regularity the dual sup supplies only as lower semicontinuity. The formalization runs on a finite window, each t handled at $T = t + 1$.

Taking the infimum over the initial coupling π turns Q_0 into the primal distance of the data, and one application of the Kantorovich–Rubinstein bridge `wasserstein1_eq_coupling` rewrites it as the dual $W_1(f_0, g_0)$ of the statement. That single `rw` is the proof’s *only* appeal to duality: the whole stability core, and the mean-field limit that follows from it, are built in the coupling cost without it.

The mean-field limit follows: applying the estimate to the Vlasov solution f and the empirical curve gives, whenever the initial empirical measures converge,

$$\sup_{t \in [0, T]} W_1(\mu_t^N, f_t) \leq e^{CT} W_1(\mu_0^N, f_0) \xrightarrow{N \rightarrow \infty} 0,$$

the separate corollary `meanFieldLimit` (Corollary 1.6).

One conjectured foundation dissolved rather than being built. The development first carried a metrization theorem relating narrow convergence and W_1 as an external gap; its single consumer needed only the easy lower-semicontinuity direction (Villani’s, with moment control [20]), a short lemma from the dual representation. Recognizing the full metrization as never load-bearing — before building it — is the endgame’s triage in action: the target theorems’ footprint was already clean of it.

3.4 The superposition principle, and why C^2

Hard: the superposition principle. Unlocked by: both — a human typeclass decision, an agent construction.

The construction of Section 3.3 produces a solution of a special form — a pushforward of the datum along the characteristic flow — and Theorem 1.3’s uniqueness lives in that Lagrangian class. Promoting it toward *all* weak solutions is the superposition principle (Theorem 1.7), and what it turns on is, once again, a *definition* — the regularity class of the potential. It costs exactly one degree beyond `Assw`; the two standing assumptions sit a single field apart:

```

class AssW (W : PhysSpace d → ℝ) : Prop where          -- C{1,1}
  differentiable : Differentiable ℝ W
  even           : ∀ x, W (-x) = W x
  lipschitzGrad  : ∃ L : NNReal, LipschitzWith L (fun x => fderiv ℝ W x)
class AssW2 (W : PhysSpace d → ℝ) extends AssW W : Prop where  -- C2, one more field
  gradContDiff  : ContDiff ℝ 1 (fun x => fderiv ℝ W x)  -- ∀W ∈ C1, i.e. W ∈ C2

```

This is the one subsection where the Lean is not confirmation standing beside the mathematics but the content itself. The proof reduces uniqueness for the nonlinear equation to uniqueness for the *linear* continuity equation with the field frozen at ρ_t — both f and the flow-pushforward solve it with the same datum — and proves the linear uniqueness by the *dual transported test function* $\psi(s, z) := \varphi(\Phi_{s \rightarrow T}(z))$, constant along characteristics. For $\psi(s, \cdot)$ to be an admissible C^1 test function the flow must be C^1 in its initial point, and that single requirement is exactly what AssW2 buys and AssW does not: at the bare $C^{1,1}$ the flow is only bi-Lipschitz, the chain-rule identity holds merely Lebesgue-a.e. rather than f_t -a.e., and the representation needs the full superposition theorem of Ambrosio [21]. (Dobrushin sidesteps the equivalence by taking the Lagrangian representation as the *definition* of solution, which is why the marquee theorems need only AssW and only this bridge pays for AssW2.)

That C^1 dependence is the *variational equation* — absent from the ambient library, supplied in house as `charFlow_hasFderivAt_in_initialPoint`: a C^1 input (the AssW2 field), a `HasFderivAt` conclusion, the regularity ladder in one signature. Formalization here did not merely *certify* a known argument; it *sharpened* it, forcing the exact degree of smoothness out of the prose and onto a type class, where a referee reads it off. The *decision* that superposition turns on exactly this, and hence on AssW2, was the mathematician’s; the *construction* — a fundamental matrix $\dot{M} = A(s, z) M$ by Picard iteration, then Fréchet differentiation under the integral — was some fifteen commits of mechanical proof the library did not shorten, and the agent carried it: the division of labor at the resolution of a single theorem.

3.5 The reusability outcome

The outcome: the layer, extracted and certified.

After the game was won, we reorganized the optimal-transport machinery into a self-contained layer (Definition 2.2), taking the layer to be the *reachable general core*: the general-mathematics declarations on a target theorem’s proof path. General infrastructure we built but did not use — a deferred truncated-metric regime and a few lemmas a refactor superseded — is excluded: a reusable layer should be what the development used, not everything general we happened to build.

The cut, read off the declaration-level dependency graph, is one-directional: the development outside the layer reaches it through a 22-declaration interface — the Wasserstein-1 and transport-cost API, the coupling predicate `IsCoupling`, and the two phase-space types — and *zero* edges run the other way. Directionality and interface width, not the size of the layer, are what make it upstreamable; the reverse-edge count is the hard fact a referee re-derives from the graph.

Checks (Q2) and (Q3) turn that graph audit into artifacts a referee re-runs, and both pass at HEAD. Packaged as a standalone Lake project, the layer builds against Mathlib alone — no kinetic-theory file in its import graph (Q2) — and passes Mathlib’s environment-linter suite (16 linters) with zero warnings (Q3). The Q3 pass is annotated: seven declarations carry disclosed `@[noLint unusedArguments]` for interface arguments retained for caller uniformity; eighteen genuinely-unused instance arguments were removed.

The whole certificate is one re-runnable block:

(Q1) reverse edges, general $\rightarrow$ specific	0
(Q2) modules' standalone build vs. Mathlib alone	exit 0
(Q3) environment linters / warnings	16 / 0
(Q4) layer reachable from a target	49/49
axiom footprint of the target theorems	[propext, Classical.choice, Quot.sound]
size: general / total declarations	49/299 $\approx$ 16%
interface width w	22

The three (Q4) sub-checks differ in strength: reachability (49/49) holds *by construction* (the layer is the reachable core); generality is *certified by* (Q2) — a layer module cannot compile against Mathlib alone if it names a Vlasov object; non-redundancy is library-search *evidence* (no **exact?** hit against the pinned Mathlib), not proof — **exact?** cannot see a semantic duplicate stated in a different form.

The reported width is the honest one, not the smallest achievable: five superseded declarations sit just outside the reachable core, and the interface names they still use raise w from a possible 17 to 22; pruning them is signature-touching — it re-triggers the full certification chain — so 22 stands. The size row, likewise, is descriptive rather than part of the certificate (Definition 2.2).

4 Assessment and outlook

4.1 The verification discipline

The discipline that keeps the game honest is a small set of standing rules, all variations on a single theme: *the machine-checkable artifact is the only certification, and an automated agent's claim of success is not one.*

1. *Read before building.* Every consequential move is preceded by reading the actual construction at the atom level, not the interface's stated summary. The phantom foundation and the tiling reach-slack (Section 3.3) were both caught by reading the construction rather than trusting a plausible summary of it.
2. *Certify by the footprint, not the build.* The win condition is the axiom footprint, and it is checked — by hand, re-running `#print axioms` — after every structural change, never inferred from a green build or an agent's report.
3. *A failed build poisons its checks.* A secondary check that ran against cached artifacts from a failed build certifies the old state; build success is confirmed before any check is trusted.
4. *Move, verify, commit, one unit at a time.* Structural changes are made one mathematical unit per step, each followed by build, footprint check, and commit, so that any regression is localized to a single, revertible move. When a move broke the build during the case study, the discipline caught it, reverted, re-diagnosed, and redid the move — and the procedure preventing that class of error was encoded into $\mathcal{G}$.

The discipline is also delegable in a specific way: the human can hand a well-specified, mechanical sub-task to a background agent, provided the human re-certifies the result by the footprint — because the gates catch a failure, but diagnosing and reverting it is a judgment the human retains. This delegation-under-recertification is the operational form of the division of labor read off the commit record next.

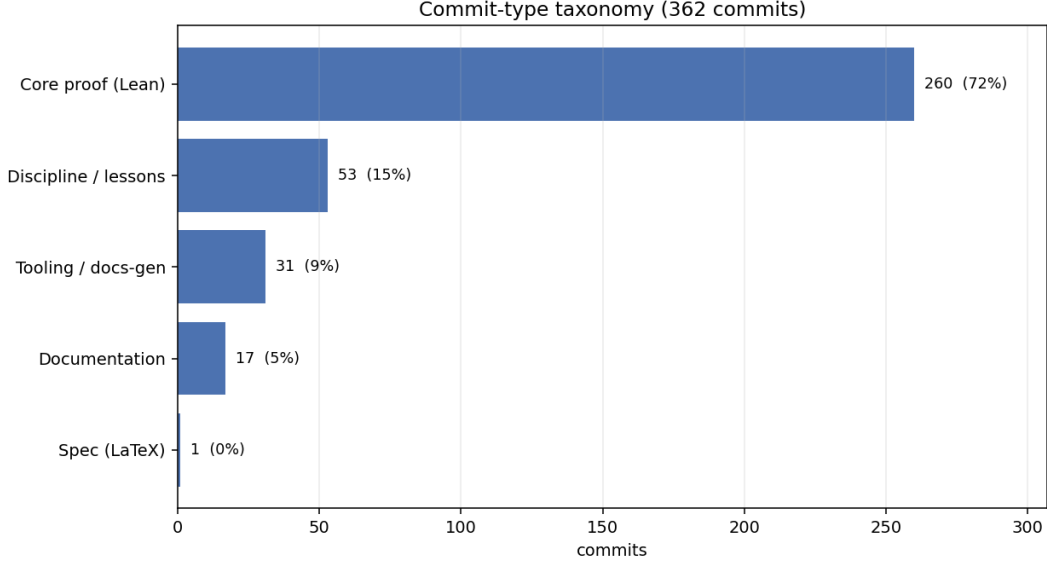


Figure 4: From the case study of Section 3: all 362 commits classified by primary intent. Roughly one in seven is pure meta-knowledge capture (discipline / lessons) — a direct, if heuristic, measure of the meta-learning loop, distinct from the proof work itself.

4.2 Timeframe and the division of labor

The well-posedness and stability marquee — from the first translated theorem to the certified stability estimate — ran in about a week; the full development, with the superposition follow-on, spanned about a month, on a flat-rate subscription — Anthropic’s \$200-per-month Max plan, no per-token billing. The claim is not raw speed but the *mode* that makes it possible: a human mathematician as strategic director over an AI execution engine, the division Section 3 marked stage by stage.

The commit record measures the split. Figure 4, the third series, classifies all 362 commits by primary intent: just under three-quarters are core proof work, but roughly one commit in seven is pure meta-knowledge capture — the share of effort that goes not into proving the next lemma but into distilling *how* it should be proved (the meta-learning loop of Section 2.3). The claim is that the division of labor, not either party alone, is what the timeframe measures; the discipline above is why the speed did not cost soundness.

We are not the only data point: an independent, contemporaneous formalization [12] — a different kinetic-theory target, the Vlasov–Maxwell–Landau equilibrium — reports a comparable mode, a single supervising mathematician completing the work in roughly ten days, at about \$200, writing no Lean by hand. Two projects are not a controlled measurement, but they make the strategic-director mode less of an outlier than a single run would suggest.

4.3 What generalizes

The three-phase structure and the strategy taxonomy (Section 2) are not specific to kinetic theory: the definitional-scoping, decomposition-steering, and library-gap-triage risks should recur in any formalization of research analysis, where the rate-limiting resource is the ambient library’s functional-analytic coverage. Of the four strategy series, the P- and B-series are domain-independent, the M-series carries analysis-flavored principles likely to recur in PDE formalization, and the L-series

is the most tool-specific and the likeliest to age with the proof assistant. The transferable artifact is Appendix B: one verbatim lesson per series, liftable into another project’s standing instructions.

4.4 Limitations

The self-contained-layer check (Definition 2.2) is pass/fail rather than a number, and we have applied it to a single development; the open question is not whether a fraction generalizes but whether its four checks (Q1)–(Q4) transfer cleanly to other projects. Even in this case the extracted layer still imports Mathlib wholesale rather than the minimal files each declaration needs, so it remains a candidate for upstreaming rather than a completed contribution. The timeframe is a single observation that depends on the human director’s prior fluency in both the mathematics and the proof assistant. The strategy taxonomy was earned on one project and may be incomplete; we expect later games to add series entries. Finally, one genuine library gap in the case study (a Polish-space completeness fact for the Wasserstein metric) was *deferred* as a cited placeholder rather than built — it lies off the path of the certified target theorems; building it remains future work, most naturally as an upstream contribution.

4.5 Outlook

The natural next edge is geometric. The Vlasov equation inherits a Hamiltonian, Lie–Poisson structure from the N -particle system — an energy $\mathcal{H}[f]$ and a bracket $\{F, G\}[f]$ for which $\partial_t f = \{f, \mathcal{H}\}$ — and identifying that bracket as the Lie–Poisson bracket on the dual of the Lie algebra of Hamiltonian vector fields, arising as the $N \rightarrow \infty$ limit of the canonical brackets on $(\mathbb{R}^d \times \mathbb{R}^d)^N$, is not formalized here; it is the subject of [28] and a natural target for a follow-on game. The present development covers the dynamics but not the geometric structure of the limit. Marking that boundary is the same statement-side discipline the paper runs on: say exactly what is proved.

Beyond the next target is a practical question: how a working mathematician should use these tools, given how fast they change. The premise is churn. The agent system named in Section 3 will be superseded, perhaps before this paper is refereed; the L-series lessons will age with the elaborator they describe; the timeframe of Section 4.2 is a single observation about tools that no longer exist in that form. Nothing in this paper should be read as advice about a particular system.

What ages more slowly is the craft. An analyst’s working knowledge, beyond the theorems, is a repertoire of moves — integrate by parts and see what the boundary term costs, split near field from far, trace a constant to see whether it is structural or an artifact — none of them theorems, all of them learned from advisors and from failure, most of them older than the problems they are currently applied to. The lessons of Section 2.3 are knowledge of the same kind, for a newer practice. They were earned the same way, and we expect them to transfer the same way: not as a method to follow but as a repertoire to draw on. We expect hybrid arrangements of this general shape — a mathematician directing, machines executing, the division shifting as the machines improve — to become a normal part of how analysis is done.

The game itself should age slowest of all, because its rules mention no particular tool and no particular division of labor. The objective, the win condition, and the layer are defined against the kernel and the ambient library; `#print axioms` does not care who made the moves. If the steering of Section 2.2 migrates from the mathematician to the machine, the game does not break — the same checks score the new arrangement. That is what the framing is for: a way to state what was attempted, what was won, and what was left behind that stays meaningful while the tools underneath it turn over.

Acknowledgements. I thank the Stanford mathematics department for its hospitality and vibrant community; Fred Rajasekaran for intriguing discussions about formalization; Eleny Ionel for organizing last quarter’s mathematics colloquium talks, which kept us current with the latest developments in AI for mathematics; and the Stanford Learning Seminar on Mathematics and Computation for its wonderful seminars and discussions.

A Formal statements

The marquee results are stated as mathematics in the introduction (Section 1.3); their verbatim Lean signatures are collected here. An ellipsis (...) marks an elided standard binder or technical hypothesis, annotated in an adjacent comment; the superposition principle’s hypotheses are reproduced complete. The assumption classes [AssW](#) and [AssW2](#) and the two faces of the Wasserstein-1 metric ([wasserstein1](#), [wasserstein1_coupling](#), bridged by [wasserstein1_eq_coupling](#)) appear in the body (Sections 3.2 and 3.4), where their exact form is part of the argument.

The *weak solution* predicate (Definition 1.1); its test class is the subject of Section 3.1:

```
def IsVlasovSolution (gradW : PhysSpace d → PhysSpace d)
  (f : ℝ → Measure (PhaseSpace d)) : Prop :=
  ∀ φ : PhaseSpace d → ℝ, ContDiff ℝ (τ : ℕ) φ → HasCompactSupport φ →
  ∀ gradXφ gradVφ : PhaseSpace d → PhysSpace d,
    (∀ z, gradXφ z = gradient (fun x => φ (x, z.2)) z.1) →
    (∀ z, gradVφ z = gradient (fun v => φ (z.1, v)) z.2) →
    WeakEvolutionEq gradW f φ gradXφ gradVφ (fun _ => 0)
```

The *Lagrangian solution* predicate (Definition 1.2):

```
def IsLagrangianVlasovSolution (gradW : PhysSpace d → PhysSpace d)
  (f : ℝ → Measure (PhaseSpace d)) : Prop :=
  IsVlasovSolution gradW f ∧
  ∃ charX charV : ℝ → PhaseSpace d → PhysSpace d,
    IsCharacteristicFlow gradW (fun t => spatialMarginal (f t)) charX charV ∧
    (∀ t, f t = Measure.map (fun z => (charX t z, charV t z)) (f 0)) ∧
    (∀ s, AEMeasurable (fun z => (charX s z, charV s z)) (f 0))
```

Global well-posedness and its per-window uniqueness (Theorem 1.3):

```

theorem vlasovWellPosedness (W : PhysSpace d → ℝ) [AssW W]
  (gradW : PhysSpace d → PhysSpace d) (hgradW : ∀ x, gradW x = gradient W x)
  (L : NNReal) (hL_gradW : LipschitzWith L gradW)
  (f₀ : Measure (PhaseSpace d)) (hf₀ : HasFiniteFirstMoment f₀) :
  ∃ f : ℝ → Measure (PhaseSpace d), f 0 = f₀ ∧
    (∀ t ∈ Set.Ici (0 : ℝ), HasFiniteFirstMoment (f t)) ∧
    (∀ T_target : ℝ, 0 < T_target →
      IsLagrangianVlasovSolutionOn gradW f T_target) ∧
    (∀ g : PhaseSpace d → ℝ, Continuous g → Bornology.IsBounded (Set.range g) →
      ContinuousOn (fun t => ∫ z, g z ∂f t) (Set.Ici 0))
theorem vlasovWellPosedness_uniqueness (W : PhysSpace d → ℝ) [AssW W]
  (gradW : PhysSpace d → PhysSpace d) (hgradW : ∀ x, gradW x = gradient W x)
  (L : NNReal) (hL : LipschitzWith L gradW) (hL_pos : (0:ℝ) < L)
  (f₀ : Measure (PhaseSpace d)) (hf₀ : HasFiniteFirstMoment f₀)
  {T : ℝ} (hT : 0 < T) (f g : ℝ → Measure (PhaseSpace d))
  (hf_init : f 0 = f₀) (hg_init : g 0 = f₀)
  ... -- finite moments on [0,T]; f, g both Lagrangian on [0,T]
  : ∀ t ∈ Set.Icc (0:ℝ) T, f t = g t

```

Dobrushin stability (Theorem 1.5):

```

theorem dobrushin (W : PhysSpace d → ℝ) [AssW W]
  (gradW : PhysSpace d → PhysSpace d) (hgradW : ∀ x, gradW x = gradient W x)
  (L : NNReal) (hL : LipschitzWith L gradW)
  (f g : ℝ → Measure (PhaseSpace d))
  (hf : IsLagrangianVlasovSolution gradW f)
  (hg : IsLagrangianVlasovSolution gradW g)
  (hf_prob : ∀ t, HasFiniteFirstMoment (f t))
  (hg_prob : ∀ t, HasFiniteFirstMoment (g t)) :
  ∃ C : ℝ, 0 < C ∧ ∀ t, 0 ≤ t →
    wasserstein1 (f t) (g t) ≤
      ENNReal.ofReal (Real.exp (C * t)) * wasserstein1 (f 0) (g 0)

```

The *mean-field limit* (Corollary 1.6):

```

theorem meanFieldLimit (W : PhysSpace d → ℝ) [AssW W] (gradW ...) (L) (hL ...)
  (f₀ : Measure (PhaseSpace d)) (hf₀ : HasFiniteFirstMoment f₀)
  (f : ℝ → Measure (PhaseSpace d)) (hf_sol : IsLagrangianVlasovSolution gradW f)
  (hf_init : f 0 = f₀)
  (X V : (N : ℕ) → ℝ → Fin N → PhysSpace d)
  (hSol : ∀ N, IsNewtonSolution N gradW (X N) (V N))
  (hInit : Filter.Tendsto
    (fun N => wasserstein1 (empiricalMeasure N (X N 0) (V N 0)) f₀) atTop (nhds 0))
  (C : ℝ) (hC : 0 < C)
  (hDobrushin : ∀ N, DobrushinStabilityEstimate
    (empiricalMeasureCurve N (X N) (V N)) f C)
  (T : ℝ) (hT : 0 < T) :
  Filter.Tendsto (fun N => ⋒ t ∈ Set.Icc 0 T,
    wasserstein1 (empiricalMeasureCurve N (X N) (V N) t) (f t)) atTop (nhds 0)

```


The *superposition principle* (Theorem 1.7), its hypotheses reproduced complete — nothing elided:

```
theorem weak_isLagrangianVlasovSolutionOn
  (W : PhysSpace d → ℝ) [AssW2 W]
  (gradW : PhysSpace d → PhysSpace d) (hgradW : ∀ x, gradW x = gradient W x)
  (L : NNReal) (hL : LipschitzWith L gradW)
  (f : ℝ → Measure (PhaseSpace d)) (T : ℝ) (hT : 0 < T)
  (hf_weak : IsVlasovSolutionOn gradW f T)
  (hf_mom : ∀ t ∈ Set.Icc (0 : ℝ) T, HasFiniteFirstMoment (f t))
  (hf_narrow : ∀ (g : PhaseSpace d → ℝ), Continuous g → HasCompactSupport g →
    ContinuousOn (fun s => ∫ z, g z ∂(f s)) (Set.Icc 0 T))
  (hf_cont : ∀ x, Continuous
    (fun t => convolveFunctionMeasure gradW (spatialMarginal (f t)) x))
  (hf_cont_deriv : ContinuousOn
    (fun p : ℝ × PhysSpace d =>
      ∫ y, fderiv ℝ gradW (p.2 - y) ∂(spatialMarginal (f p.1)))
    (Set.Icc 0 T ×s (Set.univ : Set (PhysSpace d))))
  (M_p : ℝ) (hM_p_nn : 0 ≤ M_p)
  (hM_p : ∀ t ∈ Set.Icc (0 : ℝ) T, ∫ y, ||y|| ∂(spatialMarginal (f t)) ≤ M_p)
  (hTL_PL : LocalSmallness_PL_buffer L T) : -- L · T2 < 1
  IsLagrangianVlasovSolutionOn gradW f T
```

B The standing instruction file: one lesson per series, verbatim

The strategy framework is summarized in Section 2.3 but lives, in full, in the project’s `CLAUDE.md`. One entry from each of the four series, lightly abridged, shows the form: each names a failure mode, its fix, and the generalization that makes it reusable, and each was *earned* from a specific failure during the game.

An *L-series* (Lean-idiom) lesson:

```
### L10. `ring` requires `CommRing`; additive-group goals are `abel` territory

Failure mode: writing `by ring` on a purely additive goal – a difference such as
  -a - (-b) = b - a   or   (a + b) - (a + c) = b - c
– fails with `ring made no progress` when the underlying type is an `AddCommGroup`
but not a `CommRing` (e.g. EuclideanSpace ℝ (Fin d): pointwise + and -, but no
pointwise *). The failure is opaque: the goal looks "obviously ring".

Fix: use `abel` (or `abel_nf`) for additive / vector-space / inner-product-space
goals; reserve `ring` for CommRing types (ℝ, ℂ, ℝ≥0, polynomial rings, ...).

Operational rule: at the moment you write `by ring` on a vector-valued goal,
type-check "is this a CommRing?" If not – and for phase-space differences it never
is – go straight to `abel`. Avoids the build-fail-then-fix loop.
```

A *P-series* (process) lesson:

```
### P10. Build-permits vs. audit-certifies – a green build permits many stories
```

Failure mode: after a non-trivial refactor, a clean build is necessary but not sufficient for "the refactor preserved meaning". The build certifies typechecking against the new types; it does not certify that consumers use those types for the same mathematical purpose – a predicate doing double duty can split cleanly at the signature layer while the body silently drops one use.

Operational rule: after a refactor that changes hypothesis semantics, do not certify it "complete" on a green build alone. The certifying instrument is ``#print axioms`` – a body-level audit of the axiom footprint at each consumer. Green build is a precondition, not a proof.

An *M-series* (mathematical-structure) lesson:

```
### M3/M4. Artifact smallness-constraints dissolve under the moving boundary
```

Failure mode: a smallness constraint (parameter < threshold) forced by a `*static*` construction – a parameter made to bound its own growth – reads like genuine mathematics but is an artifact of the construction, not of the theorem. A green build with the binder stripped does not certify its removal.

Diagnostic: trace the constraint to its unfold site. If it makes a construction parameter (a radius, a constant moment bound) large enough to contain its own growth → artifact: dissolve it by rebuilding on the sharp time-local bound. If it makes a contraction ratio < 1 → genuine: carry it.

Fix: when removing a constraint imposed by chaining/tiling short pieces, use `*exact*` tiling (windows of width T/N , landing on the target) – not a naive offset-drop, which leaves a data-dependent reach-slack that looks general but fails for large L . Certify by instantiating the formerly forbidden value and confirming it typechecks, not by a green build with the binder gone.

A *B-series* (bridging-architecture) lesson:

```
### B3. Conjunct shape: bounded below by the consumer, above by the producers
```

Failure mode: when strengthening a predicate with a new conjunct, reading only the consumer's need fixes the `*lower*` bound and can leave the conjunct over-strong for some producer to supply; reading only the producers can leave it under-strength.

Fix: read BOTH the consumer's genuine need and EVERY producer's available data before fixing the conjunct – it belongs at the weakest-sufficient point in that interval. The one extra read (producer capacity) catches the over-strong-but-unsupplyable shape before the edit, not mid-break.

References

- [1] J. Commelin, A. Topaz, et al., *The Liquid Tensor Experiment*, Lean 4 formalization, <https://github.com/leanprover-community/lean-liquid> (completed 2022); see also J. Commelin

- and A. Topaz, *Abstraction boundaries and spec-driven development in pure mathematics*, arXiv:2309.14870.
- [2] T. Tao, Y. Dillies, B. Mehta, et al., *Formalizing the proof of the polynomial Freiman–Ruzsa conjecture in Lean 4*, <https://github.com/teorth/pfr> (2023).
 - [3] L. Becker, M. I. de Frutos-Fernández, F. van Doorn, et al., *A blueprint for the formalization of Carleson’s theorem on the convergence of Fourier series*, arXiv:2405.06423 (2024).
 - [4] P. Massot, F. van Doorn, O. Nash, *Formalising the h-principle and sphere eversion*, in *Proc. 12th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP 2023)*; arXiv:2210.07746.
 - [5] P. Massot, *Leanblueprint*, software, <https://github.com/PatrickMassot/leanblueprint>.
 - [6] Google DeepMind, *Olympiad-level formal mathematical reasoning with reinforcement learning (AlphaProof)*, Nature (2025), <https://doi.org/10.1038/s41586-025-09833-y>.
 - [7] G. Tsoukalas, A. Kovsharov, S. Shirobokov, et al. (Google DeepMind), *Advancing mathematics research with AI-driven formal proof search*, arXiv:2605.22763 (2026).
 - [8] Harmonic, *Aristotle: IMO-level automated theorem proving*, arXiv:2510.01346 (2025).
 - [9] N. Sothanaphan, *Resolution of Erdős Problem #728: a writeup of Aristotle’s Lean proof*, arXiv:2601.07421 (2026).
 - [10] *LeanMarathon: toward reliable AI co-mathematicians through long-horizon Lean autoformalization*, arXiv:2606.05400 (2026).
 - [11] B. Alexeev, K. Barreto, Y. Li, J. D. Lichtman, L. Price, J. I. Shah, Q. Tang, T. Tao, *Primitive sets and von Mangoldt chains: Erdős problem #1196 and beyond*, arXiv:2605.00301 (2026).
 - [12] V. Ilin, *Semi-autonomous formalization of the Vlasov–Maxwell–Landau equilibrium*, arXiv:2603.15929 (2026).
 - [13] M. R. Douglas, S. Hoback, A. Mei, R. Nissim, *Formalization of QFT*, arXiv:2603.15770 (2026).
 - [14] S. Armstrong, J. Kempe, *Formalization of De Giorgi–Nash–Moser theory in Lean*, arXiv:2604.05984 (2026).
 - [15] M. Zimmer, N. Pelleriti, C. Roux, S. Pokutta, *The Agentic Researcher: a practical guide to AI-assisted research in mathematics and machine learning*, arXiv:2603.15914 (2026).
 - [16] J. Avigad, *Mathematicians in the age of AI*, arXiv:2603.03684 (2026).
 - [17] The mathlib Community, *The Lean mathematical library*, in *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP 2020)*, 367–381.
 - [18] R. L. Dobrushin, *Vlasov equations*, *Funct. Anal. Appl.* **13** (1979), 115–123.
 - [19] W. Braun, K. Hepp, *The Vlasov dynamics and its fluctuations in the $1/N$ limit of interacting classical particles*, *Comm. Math. Phys.* **56** (1977), 101–113.
 - [20] C. Villani, *Optimal Transport: Old and New*, Grundlehren der mathematischen Wissenschaften **338**, Springer, 2009.

- [21] L. Ambrosio, N. Gigli, G. Savaré, *Gradient Flows in Metric Spaces and in the Space of Probability Measures*, 2nd ed., Birkhäuser, 2008.
- [22] H. Spohn, *Large Scale Dynamics of Interacting Particles*, Springer, 1991.
- [23] F. Golse, *On the dynamics of large particle systems in the mean field limit*, in *Macroscopic and Large Scale Phenomena*, Lect. Notes Appl. Math. Mech. **3**, Springer, 2016, 1–144.
- [24] H. Neunzert, *An introduction to the nonlinear Boltzmann–Vlasov equation*, in *Kinetic Theories and the Boltzmann Equation*, Lecture Notes in Math. **1048**, Springer, 1984.
- [25] M. Hauray, P.-E. Jabin, *Particle approximation of Vlasov equations with singular forces: propagation of chaos*, Annales scientifiques de l’École normale supérieure **48** (2015), fasc. 4, 891–940.
- [26] D. Lazarovici, P. Pickl, *A mean field limit for the Vlasov–Poisson system*, Arch. Ration. Mech. Anal. **225** (2017), 1201–1231.
- [27] S. Serfaty, *Mean field limit for Coulomb-type flows*, Duke Math. J. **169** (2020), no. 15, 2887–2935.
- [28] J. K. Miller, A. R. Nahmod, N. Pavlović, M. Rosenzweig, G. Staffilani, *A rigorous derivation of the Hamiltonian structure for the Vlasov equation*, Forum Math. Sigma **11** (2023), e77.